Chapter 18. Thinking functionally

This chapter covers

- Why functional programming?
- What defines functional programming?
- Declarative programming and referential transparency
- Guidelines for writing functional-style Java
- Iteration versus recursion

You've seen the term *functional* quite frequently throughout this book. By now, you may have some ideas about what being functional entails. Is it about lambdas and first-class functions or about restricting your right to mutate objects? What do you achieve from adopting a functional style?

In this chapter, we shed light on the answers to these questions. We explain what functional programming is and introduce some of its terminology. First, we examine the concepts behind functional programming—such as side effects, immutability, declarative programming, and referential transparency—and then we relate these concepts to Java 8. In **chapter 19**, we look more closely at functional programming techniques such as higher-order functions, currying, persistent data structures, lazy lists, pattern matching, and combinators.

18.1. Implementing and maintaining systems

To start, imagine that you've been asked to manage an upgrade of a large software system that you haven't seen. Should you accept the job of maintaining such a software system? A seasoned Java contractor's only slightly tongue-in-cheek maxim for deciding is "Start by searching for the keyword `synchronized`: if you find it, just say no (reflecting the difficulty of ... Otherwise, consider the structure of the system ... le more detail in the following paragraphs. ... that as you've seen in previous chapters, Java 8's addition of streams allows you to exploit parallelism without worrying about locking, provided that you embrace stateless behaviors. (That is, functions in your stream-processing pipeline don't interact, with one function reading from or writing to a variable that's written by another.)

What else might you want the program to look like so that it's easy to work with? You'd want it to be well structured, with an understandable class hierarchy reflecting the structure of the system. You have ways to estimate such a structure by using the software engineering metrics *coupling* (how interdependent parts of the system are) and *cohesion* (how related the various parts of the system are).

But for many programmers, the key day-to-day concern is debugging during maintenance: some code crashed because it observed an unexpected value. But which parts of the program were involved in creating and modifying this value? Think of how many of your maintenance concerns fall into this category![1] It turns out that the concepts of *no side effects* and *immutability*, which functional programming promotes, can help. We examine these concepts in more detail in the following sections.

1

We recommend reading Working Effectively with Legacy Code, by Michael Feathers (Prentice Hall, 2004), for further information on this topic.

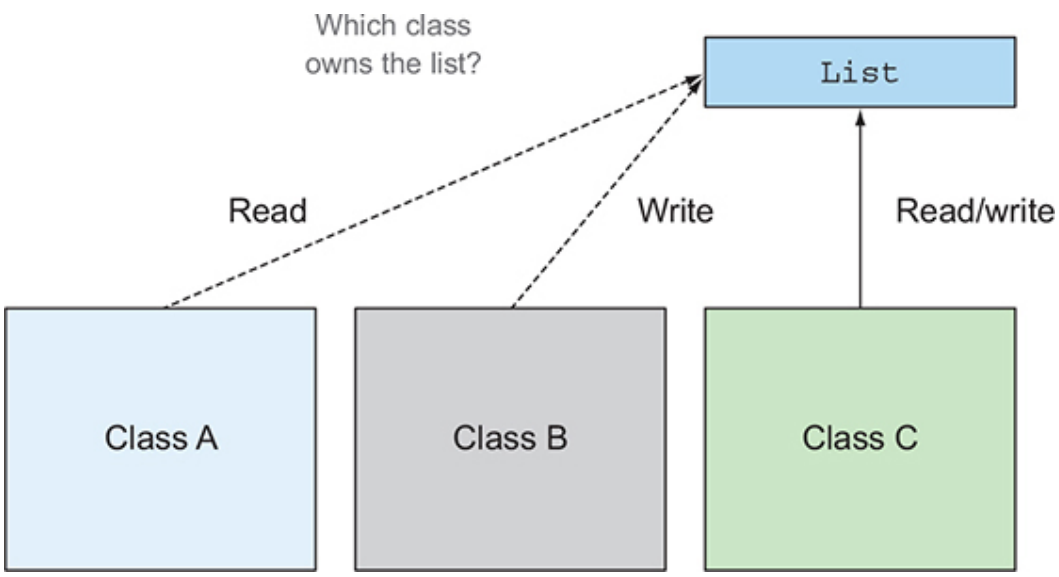
18.1.1. Shared mutable data

Ultimately, the reason for the unexpected-variable-value problem discussed in the preceding section is that shared mutable data structures are read and updated by more than one of the methods on which your maintenance centers. Suppose that several classes keep a reference to a list. As a maintainer, you need to establish answers to the following questions:

- Who owns this list?
- What happens if one class modifies the list?
- Do other classes expect this change?
- How do those classes learn about this change?
- Do the classes need to be notified of this change to satisfy all assumptions in this list, or should they make defensive copies for themselves?

utable data structures make it harder to track of your program. Figure 18.1 illustrates this

Figure 18.1. A mutable shared across multiple classes. It's difficult to understand what owns the list.



Consider a system that doesn't mutate any data structures. This system would be a dream to maintain because you wouldn't have any bad surprises about some object somewhere that unexpectedly modifies a data structure. A method that modifies neither the state of its enclosing class nor the state of any other objects and returns its entire results by using `return` is called *pure* or *side-effect-free*.

What constitutes a side effect? In a nutshell, a *side effect* is an action that's not totally enclosed within the function itself. Here are some examples:

- Modifying a data structure in place, including assigning to any field, apart from initialization inside a constructor (such as setter methods)
- Throwing an exception
- Performing I/O operations such as writing to a file

Another way to look at the idea of no side effects is to consider immutable objects. An *immutable object* is an object that can't change its state after it's instantiated, so it can't be affected by the actions of a function. When immutable objects are instantiated, they can never go into an unexpected state. You can share them without having to copy them, and they're thread-safe because they can't be modified.

The idea of no side effects may appear to be a severe restriction, and you ystems can be built this way. We hope to per- built by the end of the chapter. The good news ems that embrace this idea can use multicore locking, because the methods can no longer interfere with one another. In addition, this concept is great for immediately understanding which parts of the program are independent.

These ideas come from functional programming, to which we turn in the next section.

18.1.2. Declarative programming

First, we explore the idea of declarative programming, on which functional programming is based.

There are two ways of thinking about implementing a system by writing a program. One way centers on how things are done. (First do this, then update that, and so on.) If you want to calculate the most expensive transaction in a list, for example, you typically execute a sequence of commands. (Take a transaction from the list and compare it with the provisional most expensive transaction; if it's more expensive, it becomes the provisional most expensive; repeat with the next transaction in the list, and so on.)

This "how" style of programming is an excellent match for classic object-oriented programming (sometimes called *imperative programming*), because it has instructions that mimic the low-level vocabulary of a computer (such as assignment, conditional branching, and loops), as shown in this code:

```
Transaction mostExpensive = transactions.get(0);
if(mostExpensive == null)
    throw new IllegalArgumentException("Empty list of transactions"
for(Transaction t: transactions.subList(1, transactions.size())){
    if(t.getValue() > mostExpensive.getValue()){
        mostExpensive = t;
    }
}
```

The other way centers on what's to be done. You saw in **chapters 4** and **5** that by using the Streams API, you could specify this query as follows:

```
Optional<Transaction> mostExpensive =
                          tream()
                          ax(comparing(Transaction::getValue));
```


The fine detail of how this query is implemented is left to the library. We refer to this idea as *internal iteration*. The great advantage is that your query reads like the problem statement, and for that reason, it's immedi-

ately clear, compared with trying to understand what a sequence of commands does.

This "what" style is often called *declarative programming*. You provide rules saying what you want, and you expect the system to decide how to achieve that goal. This type of programming is great because it reads closer to the problem statement.

18.1.3. Why functional programming?

Functional programming exemplifies this idea of declarative programming (say what you want using expressions that don't interact, and for which the system can choose the implementation) and side-effect-free computation, explained earlier in this chapter. These two ideas can help you implement and maintain systems more easily.

Note that certain language features, such as composing operations and passing behaviors (which we presented in **chapter 3** by using lambda expressions), are required to read and write code in a natural way with a declarative style. Using streams, you can chain several operations to express a complicated query. These features characterize functional programming languages. We look at these features more carefully under the guise of combinators in **chapter 19**.

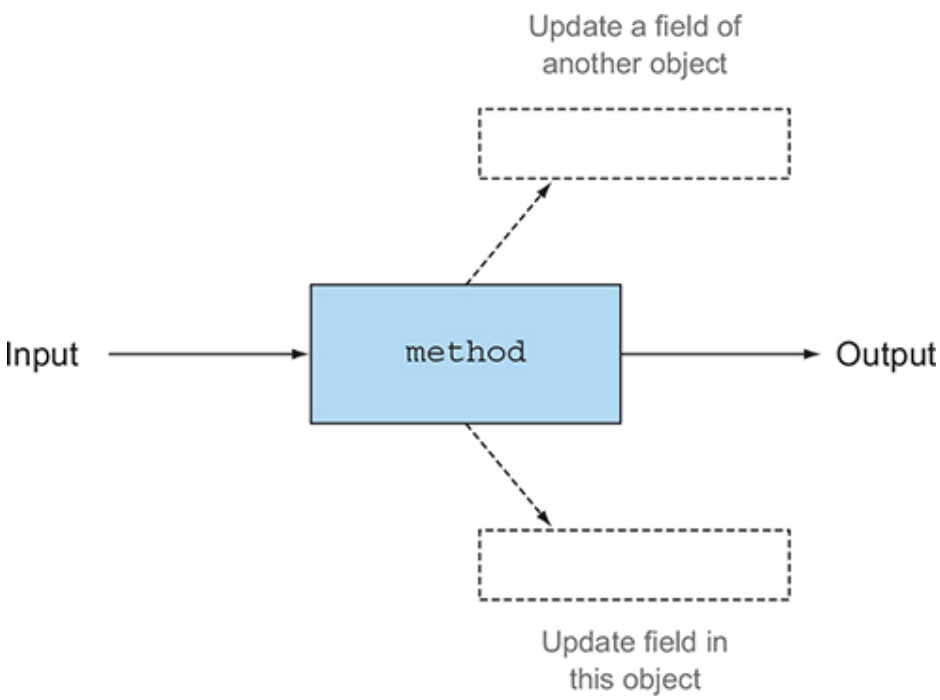
To make the discussion tangible and connect it with the new features in Java 8, in the next section we concretely define the idea of functional programming and its representation in Java. We'd like to impart the fact that by using functional-programming style, you can write serious programs without relying on side effects.

18.2. What's functional programming?

The oversimplistic answer to "What is functional programming?" is "Programming with functions." What's a function?

It's easy to imagine a method taking an `int` and a `double` as arguments -and also having the side effect of counting the en called by updating a mutable variable, as il-

Figure 18.2. A function with side effects



In the context of functional programming, however, a *function* corresponds to a mathematical function: it takes zero or more arguments, returns one or more results, and has no side effects. You can see a function as being a black box that takes some inputs and produces some outputs, as illustrated in **figure 18.3**.

Figure 18.3. A function with no side effects

The distinction between this sort of function and the methods you see in programming languages such as Java is central. (The idea that mathematical functions such as `log` or `sin` might have such side effects in unthinkable.) In particular, mathematical functions always return the same results when they're called repeatedly with the same arguments. This characterization rules out methods such as `Random.nextInt`, and we further discuss this concept of referential transparency in **section 18.2.2**.

When we say *functional*, we mean like mathematics, with no side effects. ety appears. Do we mean either: Is every func- ons and mathematical ideas such as `if-then-` do nonfunctional things internally as long as it doesn't expose any of these side effects to the rest of the system? In other words, if programmers perform a side effect that can't be observed by callers, does that side effect exist? The callers don't need to know or care, because it can't affect them.

To emphasize the difference, we refer to the former as pure functional programming and the latter as functional-style programming.

18.2.1. Functional-style Java

In practice, you can't completely program in pure functional style in Java. Java's I/O model consists of side-effecting methods, for example. (Calling `Scanner.nextLine` has the side effect of consuming a line from a file, so calling it twice typically produces different results.) Nonetheless, it's possible to write core components of your system as though they were purely functional. In Java, you're going to write functional-style programs.

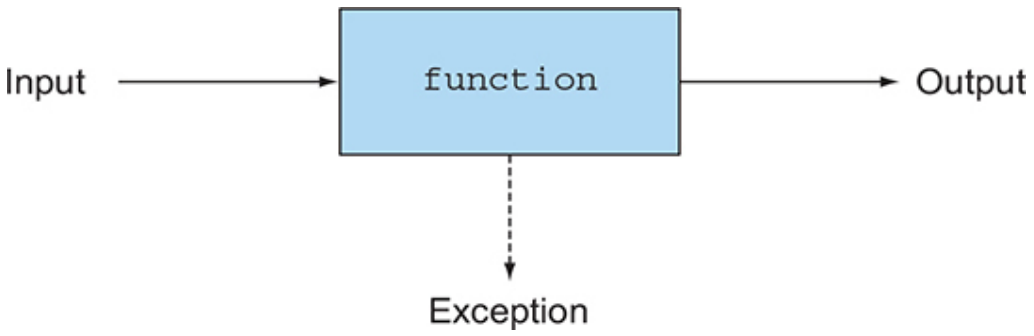
First, there's a further subtlety about no one seeing your side effects and, hence, in the meaning of *functional*. Suppose that a function or method has no side effects except for incrementing a field after entry and decrementing it before exit. From the point of view of a program that consists of a single thread, this method has no visible side effects and can be regarded as functional style. On the other hand, if another thread could inspect the field—or could call the method concurrently—the method wouldn't be functional. You could hide this issue by wrapping the body of this method with a lock, which would enable you to argue that the method is functional. But in doing so, you'd lose the ability to execute two calls to the method in parallel by using two cores on your multicore processor. Your side effect may not be visible to a program, but it's visible to the programmer in terms of slower execution.

Our guideline is that to be regarded as functional style, a function or method can mutate only local variables. In addition, the objects that it references should be immutable—that is, all fields are `final`, and all fields of reference type refer transitively to other immutable objects. Later, you may permit updates to fields of objects that are freshly created in the method, so they aren't visible from elsewhere and aren't saved to affect the result of a subsequent call.

Our guideline is incomplete, however. There's an additional requirement function or method *shouldn't throw any exceptions*. t throwing an exception would mean that a re- r than via the function returning a value; see ure 18.2. There's scope for debate here, with some authors arguing that uncaught exceptions representing fatal errors are okay and that it's the act of catching an exception that represents non-functional control flow. Such use of exceptions still breaks the simple

"pass arguments, return result" metaphor pictured in the black-box model, however, leading to a third arrow representing an exception, as illustrated in figure 18.4.

Figure 18.4. A function throwing an exception



Functions and partial functions

In mathematics, a function is required to give exactly one result for each possible argument value. But many common mathematical operations are what should properly be called partial functions. That is, for some or most input values, they give exactly one result, but for other input values, they're undefined and don't give a result at all. An example is division when the second operand is zero or `sqrt` when its argument is negative. We often model these situations in Java by throwing an exception.

How might you express functions such as division without using exceptions? Use types like `Optional<T>`. Instead of having the signature "`double sqrt(double)` but may raise an exception," `sqrt` would have the signature `Optional<Double> sqrt(double).` Either it returns a value that represents success, or it indicates in its return value that it couldn't perform the requested operation. And yes, doing so does mean that the caller needs to check whether each method call may result in an empty `Optional`. This may sound like a huge deal, but pragmatically, given our guidance on functional-style programming versus pure functional pro-

e to use exceptions locally but not expose them thereby gaining the advantages of functional ode bloat.

To be regarded as functional, your function or method should call only those side-effecting library functions for which you can hide nonfunctional behavior (that is, ensuring that any mutations they make in data struc-

tures are hidden from your caller, perhaps by copying first and by catching any exceptions). In section 18.2.4, you hide the use of a side-effecting library function `List.add` inside a method `insertAll` by copying the list.

You can often mark these prescriptions by using comments or declaring a method with a marker annotation—and match the restrictions you placed on functions passed to parallel stream-processing operations such as `Stream.map` in chapters 4–7.

Finally, for pragmatic reasons, you may find it convenient for functional-style code to be able to output debugging information to some form of log file. This code can't be strictly described as functional, but in practice, you retain most of the benefits of functional-style programming.

18.2.2. Referential transparency

The restrictions on no visible side effects (no mutating structure visible to callers, no I/O, no exceptions) encode the concept of referential transparency. A function is referentially transparent if it always returns the same result value when it's called with the same argument value. The method `String.replace`, for example, is referentially transparent because `"raoul".replace('r', 'R')` always produces the same result (`replace` returns a new `String` with all lowercase `r`s replaced with uppercase Rs) rather than updating its `this` object, so it can be considered to be a function.

Put another way, a function consistently produces the same result given the same input, no matter where and when it's invoked. It also explains why `Random.nextInt` isn't regarded as being functional. In Java, using a `Scanner` object to get the input from a user's keyboard violates referential transparency because calling the method `nextLine` may produce a different result at each call. But adding two `final int` variables always produces the same result because the content of the variables can never change.

s a great property for program understanding. nstead-of-recompute optimization for expensive or long-lived operations, a process that goes by the name *memoization* or *caching*. Although important, this topic is a slight tangent here, so we discuss it in chapter 19.

Java has one slight complication with regard referential transparency. Suppose that you make two calls to a method that returns a `List`. The two calls may return references to distinct lists in memory but containing the same elements. If these lists are to be seen as mutable object-oriented values (and therefore nonidentical), the method isn't referentially transparent. If you plan to use these lists as pure (immutable) values, it makes sense to see the values as being equal, so the function is referentially transparent. In general, in functional-style code, you choose to regard such functions as being referentially transparent.

In the next section, we explore whether to mutate from a wider perspective.

18.2.3. Object-oriented vs. functional-style programming

We start by contrasting functional-style programming with (extreme) classical object-oriented programming before observing that Java 8 sees these styles as being mere extremes on the object-oriented spectrum. As a Java programmer, without consciously thinking about it, you almost certainly use some aspects of functional-style programming and some aspects of what we'll call extreme object-oriented programming. As we remarked in **chapter 1**, changes in hardware (such as multicore) and programmer expectation (such as database-like queries to manipulate data) are pushing Java software-engineering styles more to the functional end of this spectrum, and one of the aims of this book is to help you adapt to the changing climate.

At one end of the spectrum is the extreme object-oriented view: everything is an object, and programs operate by updating fields and calling methods that update their associated object. At the other end of the spectrum lies the referentially transparent functional-programming style of no (visible) mutation. In practice, Java programmers have always mixed these styles. You might traverse a data structure by using an `Iterator` containing mutable internal state but use it to calculate, say, the sum of values in the data structure in a functional-style manner. (In Java, as dis- s can include mutating local variables.) One of nd **chapter 19** is to discuss programming tech- ures from functional programming to enable you to write programs that are more modular and more suitable for multicore processors. Think of these ideas as being additional weapons in your programming armory.

18.2.4. Functional style in practice

To start, solve a programming exercise given to beginning students that exemplifies functional style: given a `List<Integer>` value, such as {1, 4, 9}, construct a `List<List <Integer>>` value whose members are all the subsets of {1, 4, 9}, in any order. The subsets of {1, 4, 9} are {1, 4, 9}, {1, 4}, {1, 9}, {4, 9}, {1}, {4}, {9}, and {}.

There are eight subsets, including the empty subset, written {}. Each subset is represented as type `List<Integer>`, which means that the answer is of type `List<List <Integer>>`.

Students often have problems thinking how to start and need prompting[2] with the remark "The subsets of {1, 4, 9} either contain 1 or do not." The ones that don't are subsets of {4, 9}, and the ones that do can be obtained by taking the subsets of {4, 9} and inserting 1 into each of them. There's one subtlety though: we must remember that the empty set has exactly one subset—itself. This understanding gives you an easy, natural, top-down, functional-programming-style encoding in Java as follows:[3]

2

Troublesome (bright!) students occasionally point out a neat coding trick involving binary representation of numbers. (The Java solution code corresponds to 000,001,010,011,100,101,110,111.) We tell such students to calculate instead the list of all permutations of a list; for the example {1, 4, 9}, there are six.

3

For concreteness, the code we give here uses `List<Integer>`, but you can replace it in the method definitions with generic `List<T>`; then you could apply the updated `subsets` method to `List<String>` as well as `List<Integer>`.

```
static List<List<Integer>> subsets(List<Integer> list) {
    if (list.isEmpty()) {                                          1
        List<List<Integer>> ans = new ArrayList<>();
        ans.add(Collections.emptyList());
        return ans;
    }
    List<Integer> fst = list.get(0);
    List<Integer> rest = list.subList(1,list.size());
    List<List<Integer>> subAns = subsets(rest);                    2
    List<List<Integer>> subAns2 = insertAll(fst, subAns);          3
    return concat(subAns, subAns2);                                4
}
```

- ***1*** **If the input list is empty, it has one subset: the empty list itself.**
- ***2*** **Otherwise take out one element, fst, and find all subsets of the rest to give subAns; subAns forms half the answer.**
- ***3*** **The other half of the answer, subAns2, consists of all the lists in subAns but adjusted by prefixing each of these element lists with fst.**
- ***4*** **Then concatenate the two subanswers.**

The solution program produces {{}, {9}, {4}, {4, 9}, {1}, {1, 9}, {1, 4}, {1, 4, 9}} when given {1, 4, 9} as input. Do try it when you've defined the two missing methods.

To review, you've assumed that the missing methods `insertAll` and `concat` are themselves functional and deduced that your function `subsets` is also functional, because no operation in it mutates any existing structure. (If you're familiar with mathematics, you'll recognize this argument as being by induction.)

Now look at defining `insertAll`. Here's the first danger point. Suppose that you defined `insertAll` so that it mutated its arguments, perhaps by updating all the elements of `subAns` to contain `fst`. Then the program would incorrectly cause `subAns` to be modified in the same way as `subAns2`, resulting in an answer that mysteriously contained eight copies of {1,4,9}. Instead, define `insertAll` functionally as follows:

```
static List<List<Integer>> insertAll(Integer fst,
                                      List<List<Integer>> lists) {
    List<List<Integer>> result = new ArrayList<>();
    for (List<Integer> list : lists) {
        List<Integer> copyList = new ArrayList<>();            1
        copyList.add(fst);
        copyList.addAll(list);
        result.add(copyList);
    }
    return result;
}
```


can add to it. You wouldn't copy the lower-
f it were mutable. (Integers aren't mutuable.)

Note that you're creating a new `List` that contains all the elements of `subAns`. You take advantage of the fact that an `Integer` object is im-

mutable; otherwise, you'd have to clone each element too. The focus caused by thinking of methods like `insertAll` as being functional gives you a natural place to put all this carefully copied code: inside `insertAll` rather in its callers.

Finally, you need to define the `concat` method. In this case, the solution is simple, but we beg you not to use it; we show it only so that you can compare the different styles:

```
static List<List<Integer>> concat(List<List<Integer>> a,
                                  List<List<Integer>> b) {
    a.addAll(b);
    return a;
}
```

Instead, we suggest that you write this code:

```
static List<List<Integer>> concat(List<List<Integer>> a,
                                  List<List<Integer>> b) {
    List<List<Integer>> r = new ArrayList<>(a);
    r.addAll(b);
    return r;
}
```

Why? The second version of `concat` is a pure function. The function may be using mutation (adding elements to the list `r`) internally, but it returns a result based on its arguments and modifies neither of them. By contrast, the first version relies on the fact that after the call `concat(subAns, subAns2)`, no one refers to the value of `subAns` again. For our definition of `subsets`, this situation is the case, so surely using the cheaper version of `concat` is better. The answer depends on how you value your time. Compare the time you'd spend later searching for obscure bugs compared with the additional cost of making a copy.

omment that the impure `concat` is "to be used
ent can be arbitrarily overwritten, and intend-
ubsets method, and any change to `subsets`
ght of this comment," somebody sometime will find it useful in some piece of code where it apparently seems to work. Your future nightmare debugging problem has been born. We revisit this issue in **chapter 19**.

Takeaway point: thinking of programming problems in terms of function-style methods that are characterized only by their input arguments and output results (what to do) is often more productive than thinking about how to do it and what to mutate too early in the design cycle.

In the next section, we discuss recursion in detail.

18.3. Recursion vs. iteration

Recursion is a technique promoted in functional programming to let you think in terms of what-to-do style. Pure functional programming languages typically don't include iterative constructs such as `while` and `for` loops. Such constructs are often hidden invitations to use mutation. The condition in a `while` loop needs to be updated, for example; otherwise, the loop would execute zero times or an infinite number of times. In many cases, however, loops are fine. We've argued that for functional style, you're allowed mutation if no one can see you doing it, so it's acceptable to mutate local variables. When you use the `for-each` loop in Java, `for(Apple apple : apples) { }`, it decodes into this `Iterator`:

```
Iterator<Apple> it = apples.iterator();
while (it.hasNext()) {
   Apple apple = it.next();
   // ...
}
```

This translation isn't a problem because the mutations (changing the state of the `Iterator` with the `next` method and assigning to the `apple` variable inside the `while` body) aren't visible to the caller of the method where the mutations happen. But when you use a `for-each` loop, such as a search algorithm, for example, what follows is problematic because the loop body is updating a data structure that's shared with the caller:

```
public void searchForGold(List<String> l, Stats stats){
   for (String s : l){
      if("gold".equals(s)){
         stats.incrementFor("gold");
      }
   }
}
```

Indeed, the body of the loop has a side effect that can't be dismissed as functional style: it mutates the state of the `stats` object, which is shared with other parts of the program.

For this reason, pure functional programming languages such as Haskell omit such side-effecting operations. How are you to write programs? The theoretical answer is that every program can be rewritten to prevent iteration by using recursion instead, which doesn't require mutability. Using recursion lets you get rid of iteration variables that are updated step by step. A classic school problem is calculating the factorial function (for positive arguments) in an iterative way and in a recursive way (assume that the input is > 0), as shown in the following two listings.

Listing 18.1. Iterative factorial

```
static long factorialIterative(long n) {
    long r = 1;
    for (int i = 1; i <= n; i++) {
        r *= i;
    }
    return r;
}
```

Listing 18.2. Recursive factorial

```
static long factorialRecursive(long n) {
    return n == 1 ? 1 : n * factorialRecursive(n-1);
}
```

The first listing demonstrates a standard loop-based form: the variables `r` and `i` are updated at each iteration. The second listing shows a recursive definition (the function calls itself) in a more mathematically familiar form. In Java, recursive forms typically are less efficient, as we discuss immediately after the next example.

hapters of this book, however, you know that even simpler declarative way of defining factoving listing.

Listing 18.3. Stream factorial

```
static long factorialStreams(long n){
    return LongStream.rangeClosed(1, n)
                     .reduce(1, (long a, long b) -> a * b);
}
```

Now we'll turn to efficiency. As Java users, beware of functional-programming zealots who tell you that you should always use recursion instead of iteration. In general, making a recursive function call is much more expensive than issuing the single machine-level branch instruction needed to iterate. Every time the `factorialRecursive` function is called, a new stack frame is created on the call stack to hold the state of each function call (the multiplication it needs to do) until the recursion is done. Your recursive definition of factorial takes memory proportional to its input. For this reason, if you run `factorialRecursive` with a large input, you're likely to receive a `StackOverflowError`:

```
Exception in thread "main" java.lang.StackOverflowError
```

Is recursion useless? Of course not! Functional languages provide an answer to this problem: *tail-call optimization*. The basic idea is that you can write a recursive definition of factorial in which the recursive call is the last thing that happens in the function (or the call is in a tail position). This different form of recursion style can be optimized to run fast. The next listing provides a tail-recursive definition of factorial.

Listing 18.4. Tail-recursive factorial

```
static long factorialTailRecursive(long n) {
    return factorialHelper(1, n);
}
static long factorialHelper(long acc, long n) {
    return n == 1 ? acc : factorialHelper(acc * n, n-1);
}
```


`elper` is tail-recursive because the recursive
appens in the function. By contrast, in the earli-
`lRecursive`, the last thing was a multiplication of `n` and the result of a recursive call.

This form of recursion is useful because instead of storing each intermediate result of the recursion in separate stack frames, the compiler can

decide to reuse a single stack frame. Indeed, in the definition of `factorialHelper`, the intermediate results (the partial results of the factorial) are passed directly as arguments to the function. There's no need to keep track of the intermediate result of each recursive call on a separate stack frame; it's accessible directly as the first argument of `factorialHelper`. Figures 18.5 and 18.6 illustrate the difference between the recursive and tail-recursive definitions of factorial.

Figure 18.5. Recursive definition of factorial, which requires several stack frames

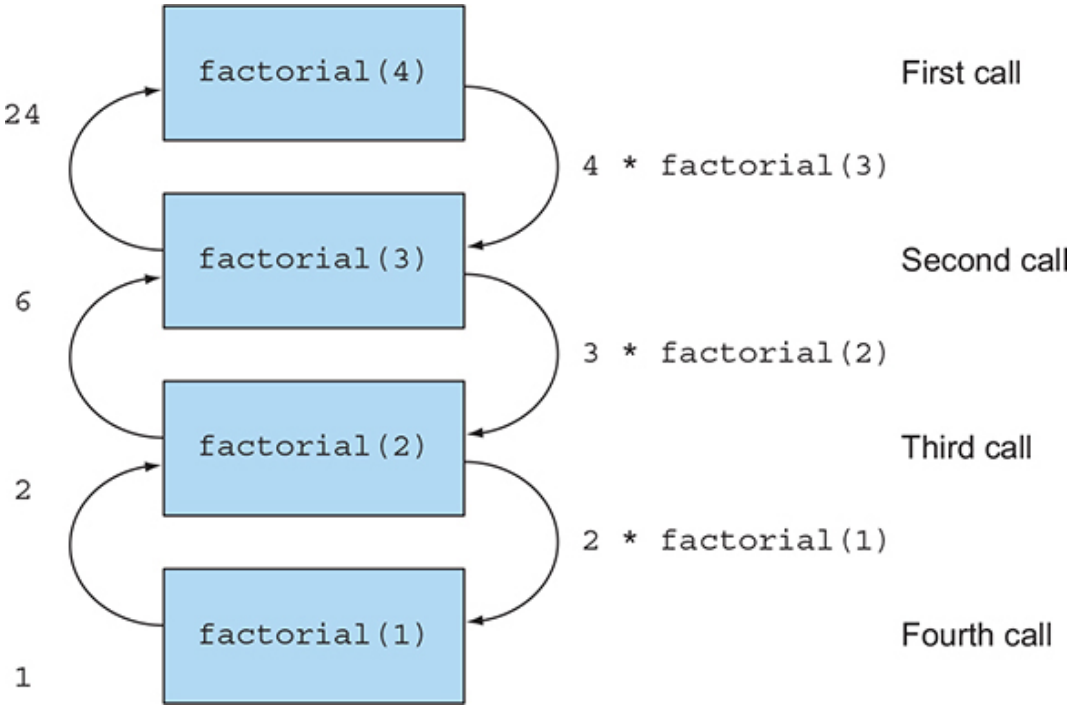
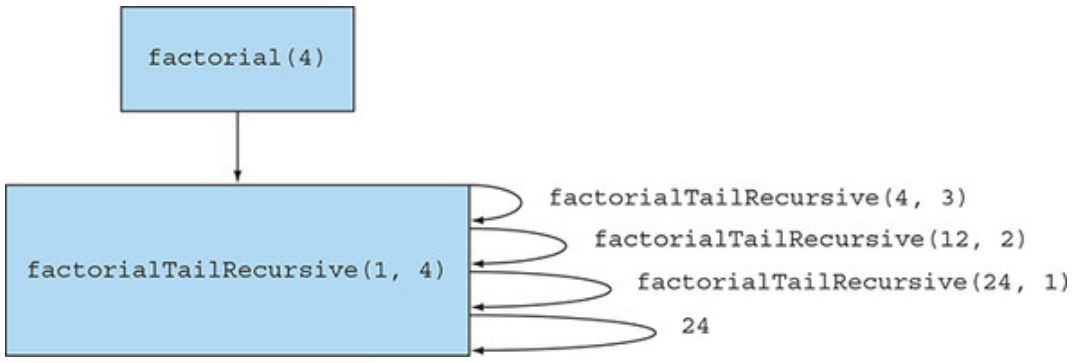


Figure 18.6. Tail-recursive definition of factorial, which can reuse a single stack frame



doesn't support this kind of optimization. But
y be a better practice than classic recursion be-
cause it opens the way to eventual compiler optimization. Many modern JVM languages such as Scala, Groovy, and Kotlin can optimize those uses of recursion, which are equivalent to iteration (and execute at the same

speed). As a result, pure-functional adherents can have their purity cake and eat it efficiently too.

The guidance in writing Java 8 is that you can often replace iteration with streams to avoid mutation. In addition, you can replace iteration with recursion when recursion allows you write an algorithm in a more concise, side-effect-free way. Indeed, recursion can make examples easier to read, write, and understand (as in the subsets example shown earlier in this chapter), and programmer efficiency is often more important than small differences in execution time.

In this section, we discussed functional-style programming with the idea of a method being functional; everything we said would have applied to the first version of Java. In **chapter 19**, we look at the amazing and powerful possibilities offered by the introduction of first-class functions in Java 8.

Summary

- Reducing shared mutable data structures can help you maintain and debug your programs in the long term.
- Functional-style programming promotes side-effect-free methods and declarative programming.
- Function-style methods are characterized only by their input arguments and their output result.
- A function is referentially transparent if it always returns the same result value when called with the same argument value. Iterative constructs such as `while` loops can be replaced by recursion.
- Tail recursion may be a better practice than classic recursion in Java because it opens the way to potential compiler optimization.